Quarto and Python notebooks

14.33 Research and Communication in Economics · Fall 2026

TL;DR: Use Quarto to share results and prepare your final reproducible paper. If you want to work in a Python notebook, I strongly recommend marimo over Jupyter. You can also work directly in Python scripts and Quarto without using a notebook.

What are these tools?

Quarto turns a text file containing prose and code into a formatted document, such as a web page or PDF. You write a `.qmd` file in a text editor. Quarto runs its code and inserts the resulting figures and tables into the document. It works for a short results update as well as a full paper.

A computational notebook is an interactive workspace where you run small blocks of code, called cells, and see their outputs alongside your notes. It is useful for exploring data or trying different sample restrictions.

marimo and Jupyter are two notebook tools. marimo provides an editor in your browser and saves your notebook as a Python (`.py`) file. Jupyter saves an `.ipynb` file containing code, document settings, and saved outputs. Both let you work cell by cell, but they handle execution and storage differently.

Why this recommendation?

Git shows changes between file versions as a line-by-line comparison, or diff. Jupyter's file format includes execution counts and saved figures, so rerunning a notebook can produce a large diff with little substantive change. marimo keeps Python source separate from outputs, as Quarto does with its `.qmd` source and rendered report. That makes edits easier to review and combine.

Readable source files also simplify work with AI coding agents, which read, edit, and execute files. marimo supports external text editors and execution as a Python script. My preference is to use these ordinary file workflows rather than require specialized Jupyter editing tools.

Jupyter also lets results depend on the order in which you happened to run cells. You might change a sample restriction but forget to rerun the regression. marimo tracks dependencies between cells: by default, running a changed cell reruns the cells that use its variables.

Share a result with Quarto

This example requires Python, Quarto, and the Python packages `matplotlib` and `jupyter`. Run commands from the folder containing your files, using your project's Python environment (its installed packages).
Save this as `quick-results.qmd`. The header between `---` lines sets document options. The text uses Markdown, a simple formatting notation. Backticks enclose the code; `{python}` tells Quarto to execute it.

````
---
title: "Weekly earnings"
format:
  html:
    embed-resources: true
jupyter: python3
---

Weekly earnings for three hypothetical workers.

```{python}
import matplotlib.pyplot as plt
hourly_wages = [18, 22, 26]
weekly_hours = 40

fig, ax = plt.subplots()
ax.bar(["A", "B", "C"], [w * weekly_hours for w in hourly_wages])
ax.set(xlabel="Worker", ylabel="Weekly earnings ($)")
plt.show()
```
````

Run `quarto render quick-results.qmd`. This executes the code and creates `quick-results.html`, which anyone can open in a browser without Python. Change `weekly_hours` and render again to update the figure. Quarto uses Jupyter to execute Python; you still write and save a `.qmd` file.

Try the same code in marimo

With the `marimo` package installed, run `marimo edit earnings.py`. In the browser editor, put the three Python lines above the blank line into one cell. Put the plotting lines into a second cell, replacing `plt.show()` with `fig`. Run both cells. Change `weekly_hours` to `30` and rerun the first cell: the figure updates too. Save the notebook as `earnings.py`. See the marimo quickstart.